

TECHNICAL ARCHITECTURE

Embedded sBTC Earn & Credit SDK + Everything Stacks

A non-custodial Bitcoin capital operating layer for sBTC deposits, yield, borrowing, swaps, portfolio intelligence, risk controls and partner distribution across the Stacks ecosystem.

SDK LAYER	PIONEER PRODUCT
Reusable APIs, protocol adapters, transaction planning, workflow orchestration, webhooks and embeddable components.	Everything Stacks proves the full user journey using exactly the same public SDK offered to external partners.

Prepared for product, engineering, protocol integration, security and ecosystem stakeholders
Version 0.1 | September 2026 | Architecture decision baseline



Document purpose and decision status

This document defines the target architecture required to move the Embedded sBTC Earn & Credit SDK and Everything Stacks from validated product concept to a working, production-capable system. It is normative for the MVP unless an architecture decision record explicitly changes a boundary, contract or release gate.

Core architectural decision

Everything Stacks is not a separate integration stack. It is the first production client of the same public SDK consumed by wallets, fintechs, exchanges, custodians and treasury platforms. This forces the SDK to be usable, observable and honest before external distribution.

Scope

- Native BTC to sBTC deposit and sBTC to BTC withdrawal.
- Supply and withdraw from verified earning markets.
- sBTC-backed USDCx borrowing, repayment and collateral management.
- Exact-input swaps through approved routes with enforced minimum output.
- Unified balances, positions, rewards, activity, performance and risk views.
- Bitcoin Staking only when a verified production capability exists.
- Public browser SDK, server SDK, React hooks, embedded UI and partner webhooks.

MVP exclusions

- No pooled strategy vault, private-key custody or automatic discretionary rebalancing.
- No guaranteed APY, principal protection or guaranteed liquidation prevention.
- No arbitrary permissionless contract routing; every executable target is registry-approved.
- No blind retry of uncertain Bitcoin or Stacks writes.

Document map

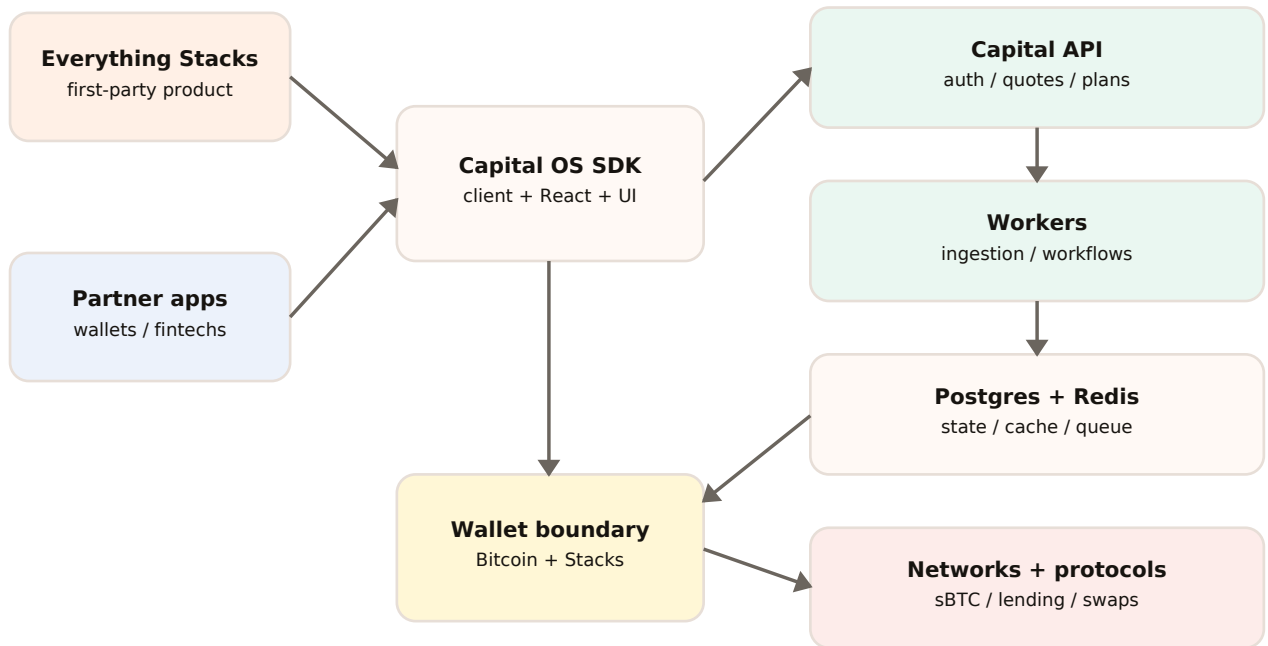
Document purpose and decision status	2
Scope	2
MVP exclusions	2
Document map	3
1. Executive architecture	6
The two product layers	6
Architecture invariants	6
Product and system boundaries	7
2. Users, customers and jobs	7
Experience promise	7
3. Trust boundaries and responsibility model	8
Custody posture	8
4. Runtime components	9
5. Repository and package architecture	9
Dependency rules	9
Recommended toolchain	10
SDK architecture	11
6. SDK product surface	11
Initialization contract	11
7. Canonical identifiers and financial types	11
Numeric rules	11
Representative canonical contracts	12
8. Capability registry and adapter boundary	12
Required adapter methods	12
9. Quote, plan and workflow architecture	13
Quote	13
Plan	13
Workflow	13
10. Public API and error contract	13
Response envelope	14
Errors drive safe UX	14
11. External developer integration lifecycle	14

Production certification	14
Everything Stacks product architecture	15
12. Product information architecture	15
Product shell	15
13. Native BTC to sBTC deposit	16
Preconditions	16
User-visible flow	16
System flow	16
Recovery	16
14. Earn supply and withdrawal	16
Opportunity comparison	16
Supply flow	16
Withdrawal flow	17
Accounting invariant	17
15. Borrow, repay and collateral management	17
Open loan	17
Critical calculations	17
Repay and exit	17
16. Swap architecture	17
MVP routing model	17
Safety requirements	18
Failure behavior	18
17. sBTC withdrawal and Bitcoin Staking	18
sBTC to native BTC	18
Bitcoin Staking	18
18. Portfolio, performance and risk	19
Freshness and confidence	19
Alerts	19
Data, security and operations	20
19. Data architecture and lineage	20
Raw to normalized pipeline	20
Reorg handling	20
20. Security architecture	21
Privacy	21
Audit scope	21

21. Testing and release gates	21
CI/CD gate order	21
Adapter release gate	21
22. Deployment, observability and incident response	22
Environment progression	22
Initial deployment shape	22
Operational signals	22
Candidate SLOs	22
23. Local bootstrap and engineering workflow	22
Install once	22
Expected repository contract	23
Definition of Done for every task	23
Business model and delivery	24
24. Business model architecture	24
Billing integrity	24
Distribution loop	24
25. PMF and adoption gates	24
North-star metrics	25
Anti-metrics	25
26. Delivery roadmap and team model	25
Two-engineer allocation	25
27. Major risks and architectural responses	26
Open decisions before production	26
28. Production acceptance checklist	26
Architecture	26
Product	26
Partner SDK	26
Operations	26
Business	27
29. Detailed engineering task allocation	28
Kenzman allocation - protocol and execution lead	28
IBK allocation - platform and product lead	29
How the split works in practice	29

1. Executive architecture

Logical system context



The architecture separates user experience, public SDK contracts, platform services, protocol-specific execution and chain truth. The SDK can read normalized markets and positions, request economic quotes, validate unsigned execution plans, coordinate wallet signing and observe workflows. It never owns unrestricted signing authority.

The two product layers

Layer	Primary user	Responsibility	Adoption effect
Capital OS SDK/API	Wallets, fintechs, exchanges, treasuries	Normalize data, construct safe transactions, persist workflows, expose components and webhooks	Every partner becomes a Stacks distribution channel
Everything Stacks	Bitcoin and Stacks users	Deliver one coherent interface for deposit, earn, borrow, swap, exit, portfolio and risk	Proves demand, creates integration evidence and generates protocol activity

Architecture invariants

- The user wallet is the signing boundary.
- Onchain state is financial truth; platform state is an evidence-backed projection.
- A quote describes economics; a plan describes an exact unsigned transaction.
- A submitted transaction is not a completed financial action.
- Every multi-step action has a persistent workflow ID and explicit next action.
- Unknown and stale data are never converted to zero or certainty.
- Protocol differences are normalized only where semantics remain valid.

PART I

Product and system boundaries

The responsibilities, trust model and runtime composition of the combined SDK and pioneer product.

2. Users, customers and jobs

Actor	Job to be done	Critical product proof
BTC holder	Use BTC productively without mastering Stacks protocol mechanics	Completes deposit, earning and exit round trip
Borrower	Access USDCx liquidity without selling BTC	Opens, monitors and closes a loan safely
Stacks DeFi user	See and operate fragmented positions from one interface	Portfolio reconciliation and repeat actions
Treasury operator	Control execution, monitor risk and preserve evidence	Policies, alerts, exports and traceable actions
Partner developer	Embed a verified Bitcoin financial flow quickly	First sandbox workflow and webhook without internal help
Partner business	Add revenue and retention without owning protocol complexity	Production integration, funded users and attributable volume
Protocol team	Gain qualified external distribution	Net-new supply, borrowing or swap volume

Experience promise

A user sees a useful Bitcoin product, while the system makes protocol identity, asset movement, fees, risk assumptions and completion state inspectable. Abstraction reduces operational complexity; it does not conceal financial risk.

Example outcomes: Earn on BTC without manually bridging and visiting several applications; borrow USDCx while retaining BTC exposure; compare supported opportunities with consistent freshness and fee treatment; return to a pending cross-chain operation without creating a duplicate transaction.

3. Trust boundaries and responsibility model

Boundary	Trusted for	Must never be trusted for
User device + wallet	Account consent, final payload review and signature	Long-lived partner secrets, server policy or canonical completion
Browser SDK	Plan validation, UX state, wallet invocation, public reads	Private keys, unrestricted broadcast or hidden contract substitution
Capital API	Authentication, quotes, plans, workflow records, partner policy	Seed phrases or claims that HTTP success equals chain truth
Adapter registry	Approved contracts, ABIs, methods, assets and capability state	Auto-enabling unverified deployments
Workers + projections	Reorg-aware history, reconciliation and notifications	Replacing direct chain evidence
RPC/indexer/oracle providers	Candidate evidence and market inputs	Sole-source certainty during disagreement, lag or outage
Protocols/signers	Protocol execution and settlement	Platform availability or invariant compatibility across versions

Custody posture

The MVP is non-custodial. The platform creates unsigned payloads and observes public state. It does not receive seed phrases or unrestricted signing authority. Any later custodial, pooled or automated strategy is a new product class requiring separate contracts, legal analysis, threat modelling and audit scope.

Cross-chain truth

Bitcoin deposit, signer processing and Stacks mint are separate states. Stacks withdrawal request, signer decision and Bitcoin payout are separate states. The product must never compress either route into a false atomic success.

4. Runtime components

Component	Core responsibility	Failure behavior
Everything Stacks web	Product shell, review UI, wallet connection, portfolio and workflow rendering	Preserve workflow IDs; show last known data with freshness
Public SDK	Typed client, runtime schemas, local validation, wallet abstraction and workflow helpers	Fail closed on schema/network/policy mismatch
Capital API	Capabilities, assets, markets, portfolios, quotes, plans, workflows, auth, alerts and partners	Return typed retry/requote/investigate errors
Quote + plan engine	Snapshot economics and build deterministic unsigned steps	Expire when market inputs or registry versions change
Protocol adapters	Protocol-native reads, quotes, calls, events, reconciliation and risk semantics	Disable action independently; preserve safe reads/exits
Ingestion worker	Blocks, raw events, canonical projections and checkpoints	Rewind and rebuild on reorg
Workflow worker	Observe transaction lifecycle and derive next action	Never blindly resubmit uncertain writes
Webhook worker	Signed delivery, retry, deduplication and dead-letter visibility	Delivery failure cannot change onchain state
PostgreSQL	Durable platform state, evidence lineage, audit and usage	Backups, point-in-time recovery and migration controls
Redis/queue	Cache, locks, rate limits and scheduled work	Loss cannot erase canonical workflows

5. Repository and package architecture

apps/	
web/	Everything Stacks Next.js client
api/	versioned REST API and authentication
worker/	ingestion, reconciliation, workflows, alerts
partner-example/	clean external integration
packages/	
core/	schemas, AssetId, bigint/fixed-point, errors
adapters/	interface + versioned protocol implementations
sdk/	browser client, plan validation, workflows
server-sdk/	API-key and webhook helpers
wallets/	Bitcoin and Stacks capability interfaces
react/	providers, cache and workflow hooks
ui/	optional white-label components
config/	signed capability/deployment registry
fixtures/	deterministic contract and failure scenarios
observability/	logs, metrics and trace helpers

Dependency rules

- Adapters cannot import product UI.
- The web application cannot import internal transaction builders; it calls the public SDK.
- Browser packages cannot import partner secrets, Node-only modules or infrastructure credentials.
- Runtime schemas generate or validate OpenAPI and SDK types; types alone are insufficient at trust boundaries.
- Changesets version packages independently inside a tested compatible release set.

Recommended toolchain

Area	Choice	Reason
Language	Strict TypeScript	One contract across web, SDK, API, workers and adapters
Workspace	pnpm workspace + reproducible lockfile	Shared packages without implicit dependency drift
Web	Next.js + React Query/state isolation	Product and embedded component compatibility
API	Versioned REST + generated OpenAPI	Broad partner compatibility and contract testing
Data	PostgreSQL + Redis/queue	Durable evidence plus short-lived coordination
Validation	Runtime schemas + bigint/fixed decimal	Safe external data and exact financial arithmetic
Local infra	Docker Compose	Reproducible database, cache and worker development

PART II

SDK architecture

The public integration surface, canonical contracts, adapters, quotes, plans, workflows and external developer lifecycle.

6. SDK product surface

Package	Environment	Exports
@capital-os/core	Browser + server	Assets, amounts, capabilities, errors, schemas, plan validation
@capital-os/client	Browser + server	Markets, portfolios, quotes, plans, workflows and alerts client
@capital-os/wallets	Browser	Bitcoin/Stacks wallet capability interfaces and request renderers
@capital-os/react	Browser	Provider, address-scoped cache, market/portfolio/workflow hooks
@capital-os/ui	Browser	White-label Deposit, Earn, Borrow, Swap, Progress and Risk components
@capital-os/testing	Test	Mock transport, fixtures, fault injection and sandbox helpers
@capital-os/server	Server only	API-key auth, webhook verification, usage and reporting helpers

Initialization contract

```
const capital = createCapitalClient({
  environment: 'sandbox',
  publicClientId: process.env.NEXT_PUBLIC_CAPITAL_CLIENT_ID!,
  network: 'testnet',
  wallet: walletAdapter,
});
```

Initialization retrieves signed, versioned capability metadata. Applications render unavailable actions from runtime capability results rather than hardcoding protocol support.

7. Canonical identifiers and financial types

Type	Required identity	Key rule
AssetId	Chain, network, native-or-contract principal, asset name	Ticker alone is never an identifier
DeploymentId	Protocol, network and contract revision	Every executable plan pins a revision
MarketId	Stable platform ID mapped to deployment/capability set	Survives UI naming changes
PositionId	Owner, deployment, market, position kind and protocol key	Supports reconciliation and deduplication
WorkflowId / StepId	Immutable platform execution identity	Transactions are references, not workflow IDs

Numeric rules

- Token quantities cross JSON as base-10 integer strings and use bigint internally.
- Prices and rates use fixed decimal values with currency, scale, source, timestamp and block context.
- No JavaScript number participates in signing, collateral, debt, minimum-output or accounting arithmetic.

- Every adapter documents rounding direction for supply, redeem, borrow, repay, collateral and fee calculations.

Representative canonical contracts

```
type DataPoint<T> = {
  value: T | null; observedAt: string; blockHeight?: number;
  blockHash?: string; source: string; stale: boolean; warnings: string[];
};

type Quote = {
  id: string; action: Action; marketId: string;
  input: AssetAmount[]; expectedOutput: AssetAmount[];
  fees: Fee[]; minimumOutput?: AssetAmount;
  snapshots: string[]; expiresAt: string; executable: boolean;
  warnings: string[];
};
```

8. Capability registry and adapter boundary

An adapter is executable financial integration code, not a logo or market-data connector. Every supported action is the intersection of a versioned adapter, verified deployed target, exact asset pair, wallet capability, operational status and feature policy.

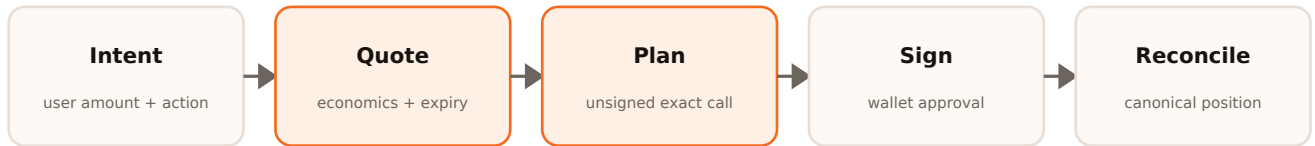
Registry field	Purpose
Network + protocol + adapter version	Resolve exact implementation
Contract/ABI revision	Prevent silent target substitution
Action + exact asset IDs	Constrain supply, withdraw, borrow, repay, swap or stake
Wallet requirements	Block unsupported signing routes before review
State + reason	Enabled, read-only, paused or disabled
verifiedAt + audit references	Expose provenance and review status
Feature flag + tenant policy	Allow independent emergency and partner control

Required adapter methods

Method	Contract
describeCapabilities	Static and runtime action support
listMarkets / getMarket	Normalized identity, rates, liquidity, caps and state
readPositions	Protocol-native position reads with block reference
quote	Economic result and prerequisites without mutation
buildPlan	Deterministic unsigned plan bound to quote, registry and network
validatePlan	Independent asset, contract, method, amount and post-condition invariants
decodeEvents	Raw event to canonical activity
reconcile	Compare events, direct reads and projected state
explainRisk	Disclosures, variables, staleness and calculation metadata

9. Quote, plan and workflow architecture

Execution contract



Rule: quotes do not sign; plans bind quote, network, registry and adapter versions.

Rule: submitted is not complete; completion requires canonical evidence and reconciliation.

Rule: uncertain writes are inspected, never blindly retried.

Quote

Captures input/output, rates, fees, liquidity, price and protocol snapshots, expiry, warnings and whether the action is currently executable. It is informational and never opens a wallet.

Plan

Contains ordered steps, dependencies, exact unsigned payload, network, contract, method, post-conditions, expected asset effects, human review summary, expiry and adapter/registry versions. Client validation re-checks partner policy and maximum input before wallet invocation.

Workflow

Persists the multi-step lifecycle before signing. Each transition records prior/new state, reason, actor, evidence and time. Idempotency prevents duplicate workflow creation but does not authorize duplicate blockchain submission.

State group	States	Typical next action
Preparation	DRAFT, QUOTED, AWAITING_SIGNATURE	SIGN or REQUOTE
Network	BROADCAST_UNKNOWN, SUBMITTED, CONFIRMING	WAIT or RETRY_READ
Settlement	STEP_CONFIRMED, RECONCILING	WAIT
Intervention	ACTION_REQUIRED, REORGED, MANUAL_REVIEW	REQUOTE, RECLAIM or CONTACT_SUPPORT
Terminal	COMPLETED, EXPIRED, USER_REJECTED, FAILED	COMPLETE or begin a new workflow

10. Public API and error contract

Resource	Representative endpoints
Discovery	GET /v1/capabilities, /assets, /markets
Portfolio	GET /v1/addresses/{address}/positions, /risk
Execution	POST /v1/quotes, /plans, /workflows
Workflow	GET /v1/workflows/{id}, /next; POST /transactions
Identity	POST /v1/auth/challenge, /verify
Partners	POST /v1/webhook-endpoints; GET/POST /v1/alerts

Response envelope

```
{
  "schemaVersion": "1.0",
  "requestId": "req...",
  "network": "stacks:testnet",
  "data": {},
  "context": {
    "blockHeight": 123, "blockHash": "0x...",
    "observedAt": "2026-09-09T12:00:00Z",
    "stale": false, "warnings": []
  }
}
```

Errors drive safe UX

Class	Examples	Client behavior
Retryable read	PROVIDER_TIMEOUT, RATE_LIMITED	Retry with backoff; retain last known data
Requote	QUOTE_EXPIRED, CAP_REACHED, ORACLE_STALE	Invalidate plan and request fresh economics
User/action	USER_REJECTED, INSUFFICIENT_BALANCE, NETWORK_MISMATCH	Return to editable step
Investigation	BROADCAST_UNKNOWN, REORG_DETECTED, RECONCILIATION_MISMATCH	Preserve references; inspect before any write

11. External developer integration lifecycle

Stage	Developer action	Platform proof
1. Register	Create project, environments, origins, scopes and webhooks	Tenant policy and scoped credentials
2. Initialize	Install packages and create client	Signed capability metadata loaded
3. Discover	List markets and exact asset/action support	Freshness and semantic differences visible
4. Portfolio	Read address positions; optionally authenticate settings	Cache and privacy partitioned by tenant/network/address
5. Quote	Submit action, market and exact atomic amount	Economics, warnings and expiry returned
6. Plan	Build and validate unsigned transaction steps	Network, allowlist, amount and effects verified
7. Workflow	Persist plan with idempotency key	Resumable workflow created before signature
8. Sign	Render review and call wallet adapter	User approves exact chain request
9. Record	Send txid/attempt reference	No key material reaches backend
10. Observe	Hook, polling or HMAC webhook	Canonical transitions and delivery evidence
11. Resume	Render nextAction after refresh or failure	No duplicate write; stale economics requested

Production certification

- Clean Next.js/Vite installation; no server secret in browser bundle.
- Network switching invalidates quotes and plans.
- Review screen matches wallet payload and expected asset effects.
- Deposit/withdraw and earn/withdraw workflows resume after reload.
- Webhook HMAC, timestamp tolerance and event-ID deduplication pass.
- Privacy, risk, rate limits, support ownership and incident contacts are approved.

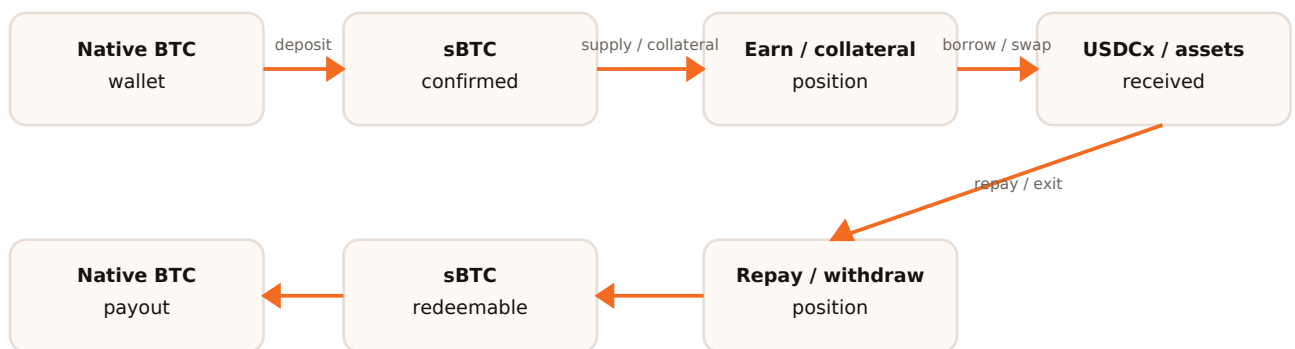
PART III

Everything Stacks product architecture

The first-party user experience that exercises the complete SDK and proves every capital journey.

12. Product information architecture

Everything Stacks capital journey



Each arrow is an independently observed workflow; Bitcoin and Stacks legs are never presented as atomic.

Route	Primary job	SDK capabilities used
Overview	See total assets, liabilities, net value, alerts and pending actions	portfolio, risk, workflows
Earn	Compare and supply sBTC; withdraw a position	markets, quote, plan, workflow, position
Borrow	Open, monitor, repay and close sBTC-backed USDCx debt	collateral, borrow, repay, risk
Swap	Execute exact-input exchange through approved route	route quote, min output, plan, activity
Positions	Inspect protocol-native and normalized state	positions, rewards, cash flows
Risk	View health, oracle, stablecoin, protocol and concentration exposure	risk, freshness, alerts
Activity	Track every cross-chain and onchain state transition	workflows, transactions, recovery
Settings	Wallets, alerts, privacy, network and transaction preferences	auth, alerts, wallet capabilities

Product shell

The header always exposes active network, connected wallet/address, data freshness and pending-action indicator. Every write opens a common transaction review drawer, while Activity renders the durable workflow timeline. This prevents each protocol integration from inventing a different safety model.

13. Native BTC to sBTC deposit

Preconditions

- Compatible Bitcoin wallet/signing path and matching Bitcoin network.
- Deposit service and signer status healthy; amount above minimum; BTC covers amount plus miner/signer fees.

User-visible flow

1. Enter BTC amount and Stacks recipient.
2. Review miner fee, maximum signer fee, expected sBTC, recipient and reclaim explanation.
3. Approve Bitcoin transaction in wallet.
4. Track Bitcoin confirmation, signer processing, Stacks mint and final balance reconciliation.

System flow

Step	Persisted state	Evidence
Build deposit	DRAFT -> AWAITING_BITCOIN_SIGNATURE	Verified library/registry version, scripts and recipient
Wallet result	SUBMITTED_BITCOIN or USER_REJECTED	txid or rejection
Bitcoin finality	BITCOIN_CONFIRMING	block/confirmation evidence
Signer processing	SIGNER_PROCESSING / DELAYED	deposit-service status
Stacks mint	MINT_OBSERVED	canonical mint event
Reconcile	COMPLETED	exact sBTC balance delta and workflow linkage

Completion rule

Only a canonical sBTC mint plus balance reconciliation permits Complete. A Bitcoin transaction, signer acknowledgement or HTTP response alone is insufficient.

Recovery

On wallet or broadcast timeout, inspect the transaction and UTXO before retry. Delayed signer processing remains pending. Rejected/unprocessed deposits surface protocol reason and reclaim eligibility. Reclaim material must survive browser storage loss under an explicitly threat-modelled storage policy.

14. Earn supply and withdrawal

Opportunity comparison

Cards expose protocol, exact supplied and receipt assets, base variable APY, incentives, net-estimate method, capacity, withdrawal liquidity, fees, freshness and risk disclosures. Markets are compared only where action and accounting semantics are genuinely comparable.

Supply flow

1. Filter supported sBTC markets and choose amount.
2. Request a quote using fresh rate/liquidity/capacity snapshots.
3. Review exact outgoing sBTC, expected receipt/position, post-conditions, fees and expiry.
4. Sign Stacks transaction; observe confirmation and protocol event.
5. Reconcile direct protocol read and portfolio projection.

Withdrawal flow

Refresh position and liquidity before plan creation. For withdraw-all, use protocol-native receipt/max semantics rather than a stale display amount. Enforce any minimum output and reconcile wallet balance, remaining position and realized cash flow.

Accounting invariant

No double counting

A receipt token that represents a supplied claim is not added on top of the underlying economic exposure. Portfolio value counts the claim once and preserves its protocol-native representation for audit.

15. Borrow, repay and collateral management

Open loan

1. Choose sBTC collateral amount or an existing eligible supplied position.
2. Enter desired USDCx debt and retrieve protocol-specific risk inputs.
3. Review collateral value, maximum borrow, selected buffer, projected health, liquidation assumptions, variable APR, fees and available liquidity.
4. Sign each explicit step. If collateral deposit and borrow are separate, the UI shows two separate signatures and outcomes.

Critical calculations

Input	Requirement
Collateral and debt	Exact base units; current accrued debt
Oracle	Protocol source, decimals, observed time and staleness
Thresholds	Exact collateral factor/liquidation threshold and rounding
Liquidity and caps	Fresh borrow liquidity, market cap and pause state
Projected health	Calculated from the exact planned post-state

Repay and exit

Partial repayment shows resulting debt and health. Repay-all uses protocol-correct maximum semantics including accrued interest. Collateral removal is built only when the protocol is expected to accept it. If repayment confirms but collateral withdrawal fails, the system preserves the repaid state and offers a fresh follow-up action.

Partial completion is a valid state

Collateral confirmed while borrow fails produces a collateral-only position. The product offers Retry Borrow or Withdraw Collateral after a fresh read; it never labels the whole workflow failed and discards the confirmed step.

16. Swap architecture

MVP routing model

Support exact-input swaps through approved routing adapters. The quote returns route legs, pools, expected output, minimum received, price impact, router/protocol fees, network fee and expiry.

Safety requirements

- Input/output assets and route targets are allowlisted by network and contract identity.
- User slippage stays within configured bounds and produces an explicit minimum output.
- The contract call or router semantics enforce the minimum output; a UI-only number is not protection.
- Expired quotes cannot become plans; changed liquidity invalidates the plan.
- Completion requires confirmed asset deltas consistent with route constraints.

Failure behavior

Condition	Response
No route / insufficient liquidity	No executable plan
Quote expired	Requote before wallet invocation
Minimum output violation	Onchain abort; preserve input-state evidence
Unsupported token/contract	Capability error before signing
Broadcast unknown	Inspect nonce/transaction before any retry

17. sBTC withdrawal and Bitcoin Staking

sBTC to native BTC

1. Validate Bitcoin destination type/network, minimum amount and fee allowance.
2. Review sBTC burn/lock request, maximum fee, estimated BTC received and multi-block timing.
3. Sign the Stacks withdrawal request.
4. Track Stacks confirmation, signer accept/reject, Bitcoin payout broadcast and BTC confirmation.

The destination cannot be edited after submission. The workflow completes only after signer acceptance and the Bitcoin payout transaction are observed, not at the Stacks request alone.

Bitcoin Staking

Staking is a distinct capability, not an alternative label for sBTC DeFi. A staking adapter must verify PoX/deployment revision, committed asset, signing path, delegation model, lock period, activation, capacity, reward source and exit behavior. If any required fact is unverified, both SDK and product return Not available with no simulated transaction.

Release independence

Staking uncertainty does not block sBTC Earn, Borrow or Swap. Capability flags allow the verified surface to launch while the unverified action remains disabled.

18. Portfolio, performance and risk

View	Contents	Data contract
Portfolio	Wallet assets, supplied claims, debt, LP exposure, staking, rewards and pending workflows	Gross assets, liabilities and net value with timestamps
Performance	Confirmed cash flows, realized yield, estimated accrual and incentive value	Known fees/interest deducted; missing cost basis = unavailable
Borrow risk	Health, thresholds, oracle inputs, liquidation assumptions and stress views	Exact adapter semantics and fresh protocol inputs
Platform risk	Protocol, stablecoin, oracle, liquidity and concentration exposure	Disclosed model, version and limitations
Activity	Every workflow, step, transaction, confirmation, reorg and recovery action	Immutable evidence-linked timeline

Freshness and confidence

Every material datapoint carries observed time, source, optional block height/hash, stale flag and warnings. Provider disagreement, index lag or reconciliation mismatch is a first-class product state. The UI can retain reliable history and safe exits while disabling decisions that require freshness.

Alerts

Alerts are advisory observations, not liquidation protection. Each notification states the threshold, observed value, timestamp, source and delivery delay. Rules are address/tenant scoped and audited.

PART IV

Data, security and operations

The evidence model, ingestion, risk controls, testing, deployment and operating discipline required for financial workflows.

19. Data architecture and lineage

Domain	Core entities
Registry	protocols, deployments, assets, capabilities, markets, disclosures, adapter_releases
Chain evidence	chain_blocks, raw_events, ingestion_checkpoints, canonical_activities, reconciliation_runs
Financial projections	balance, position, price, rate, liquidity and reward snapshots; cash_flows
Execution	quotes, plans, workflows, steps, transaction_attempts, transitions, recovery_actions
Identity/commercial	partners, apps, keys, origins, sessions, webhooks, alerts, usage_events, audit_log

Raw to normalized pipeline

1. Fetch blocks/events with provider identity and checkpoint.
2. Store immutable raw payload before interpretation.
3. Decode using the exact deployment adapter revision.
4. Project canonical activity, balances and positions.
5. Periodically compare projections with direct contract reads.
6. Attach freshness, source, adapter and calculation versions.
7. Serve normalized API with warnings rather than silent gaps.

Reorg handling

Mark orphaned blocks/events noncanonical, rewind the checkpoint, rebuild affected projections, re-evaluate workflow completion and emit a reorg transition. A previously completed workflow may temporarily re-enter REORGED or RECONCILING until new canonical evidence stabilizes.

20. Security architecture

Threat	Primary controls
Malicious/substituted plan	Signed registry, allowlists, quote binding, client validation and wallet review
Duplicate financial action	Workflow idempotency, nonce/UTXO inspection and no blind write retry
Network or asset confusion	Canonical AssetId, explicit network, contract revision and post-conditions
Stale oracle/rate/liquidity	Timestamped inputs, adapter thresholds and action-level fail closed
Cross-tenant data leak	Tenant-scoped auth, cache keys, row-level query constraints and audit
Webhook spoof/replay	HMAC, timestamp tolerance, event ID deduplication and key rotation
Partner secret in browser	Public client ID/browser policy; server keys restricted to server SDK
Registry compromise	Reviewed signed release, least privilege, emergency disable and audit history
Dependency/supply-chain attack	Lockfile, provenance, scanning, minimal publish contents and release gates

Privacy

Public address lookup is configurable and disclosed. Saved workflows, alerts and personal settings require domain-bound nonce authentication. Logs avoid unnecessary full addresses or raw payloads; analytics requires consent and cannot contain secrets. Retention and deletion policy must distinguish public chain evidence from offchain personal settings.

Audit scope

Production review covers transaction construction, post-conditions, registry signing/update, authentication, tenant boundaries, webhook verification, amount arithmetic, adapter event decoding, workflow recovery and any storage of reclaim material.

21. Testing and release gates

Layer	Required tests
Unit	Amounts, fixed decimals, rounding, IDs, error mapping, risk arithmetic and state transitions
Adapter fixtures	Contract reads, event decoding, transaction bytes, post-conditions, pauses, caps and oracle staleness
Integration	API/database/queue, providers, direct reconciliation, auth, webhooks and migrations
Wallet	Leather/Xverse supported calls, rejection, switch, timeout and unknown broadcast
End to end	Deposit, earn, withdraw, borrow, repay, swap, exit and reload recovery
Fault injection	Provider lag, reorg, signer delay, duplicate webhook, quote expiry and transaction abort
Security	Dependency, secret, authorization, tenant isolation and transaction invariant tests
Accessibility	Keyboard, screen reader, focus, contrast, responsive and error-state navigation

CI/CD gate order

Format/lint -> typecheck -> unit -> adapter fixtures -> integration -> OpenAPI compatibility -> browser bundle secret check -> dependency/security scan -> build -> ephemeral environment -> end-to-end smoke -> approval -> immutable promotion.

Adapter release gate

- Contract, ABI, address and network independently reviewed.
- Fixture reads and live test environment agree.
- All supported calls pass amount, network and asset boundary tests.

- Events reconcile with direct reads; pause/cap/liquidity/oracle failures are exercised.
- Feature flag disables new actions without removing reliable reads and safe exits.

22. Deployment, observability and incident response

Environment progression

Environment	Purpose	Restrictions
Local	Deterministic fixtures and optional devnet	No implicit mainnet; fail on missing network
Sandbox	Stable fake markets and controllable failures	No financial value; versioned fixtures
Testnet	Live wallet/RPC behavior and verified test deployments	Only registered deployments
Production	Reviewed mainnet registry, separate data and credentials	Explicit feature flags and release approval

Initial deployment shape

Deploy web, API and worker as independently scalable immutable containers with managed PostgreSQL, managed Redis, secrets manager and centralized telemetry. Avoid premature microservices. Database migrations remain backward-compatible and queue payloads tolerate rolling versions.

Operational signals

Signal	Examples
API	Availability, p50/p95/p99 latency, error class, rate limiting
Data	Head lag, stale snapshots, provider disagreement, reconciliation mismatch
Workflow	Time in state, stuck rate, broadcast unknown, signer delay, completion rate
Business	Funded wallets, completed round trips, partner-sourced volume, paid usage
Security	Auth failures, scope denial, registry changes, webhook verification failure

Candidate SLOs

- Read API monthly availability and latency target measured separately from external chain latency.
- Workflow state observation target measured from detectable provider evidence.
- Webhook delivery target measured to first verified attempt; dead letters visible to partners.
- Zero accepted plans targeting disabled/unregistered contracts.

Final numeric SLOs are set only after baseline load and provider testing. Incident severity considers funds at risk, incorrect financial information, privacy exposure and breadth of action disablement.

23. Local bootstrap and engineering workflow

Install once

- Git and repository access; Node version manager; Corepack/pnpm pinned by the repository.
- Docker Desktop/Engine with Compose for PostgreSQL and Redis.
- Leather and Xverse test wallets; never reuse production seed phrases.
- Optional Clarinet for contract fixtures/local devnet; API client plus curl/jq.

Expected repository contract

```
corepack enable
pnpm install --frozen-lockfile
docker compose up -d postgres redis
pnpm db:migrate
pnpm fixtures:seed
pnpm dev

pnpm lint
pnpm typecheck
pnpm test:unit
pnpm test:integration
pnpm test:e2e
pnpm openapi:check
```

Definition of Done for every task

- Acceptance criteria and linked requirement IDs satisfied.
- Tests and evidence linked; PR reviewed by the other engineer.
- No unresolved network, asset, amount, signing or recovery ambiguity.
- Documentation and observability updated with the code path.

PART V

Business model and delivery

How the architecture creates distribution, revenue and measurable product-market fit while moving from feasibility to production.

24. Business model architecture

Offer	Customer	Revenue model	Meter
Everything Stacks discovery	End users	Free	Connected/funded wallet and retained use
Execution orchestration	End users where permitted	Transparent action/volume fee	Reconciled eligible volume only
Launch SDK	Wallet/fintech	Platform minimum + usage	Requests, active funded wallets, completed workflows
Growth SDK	Scaled partner	Volume/revenue share + higher support	Supply, borrow, swap and workflow usage
Enterprise	Exchange/custodian/treasury	Annual contract + SLA + implementation	Dedicated controls, reporting and support
Risk/treasury module	Professional operator	Subscription	Addresses/assets monitored, exports and alerts

Billing integrity

- Usage events are immutable, tenant/environment scoped and idempotent.
- Billable onchain volume comes only from reconciled canonical transactions.
- Monitored TVL is distinct from capital attributable to the platform.
- Protocol referral and sponsored placement are disclosed and cannot override safety or objective ranking.

Distribution loop

Everything Stacks creates real workflows, telemetry and user learning -> the SDK packages those verified capabilities -> partners expose them to existing audiences -> partner activity creates demand for sBTC, USDCx, lending and swaps -> broader usage justifies additional adapters and enterprise tooling.

25. PMF and adoption gates

Gate	Evidence required	Architecture signal
Problem	15+ interviews across wallets, holders and treasuries	Repeated job and current workaround
Technical	3 partners complete sandbox validation	Time to quote/workflow, integration failures
Distribution	1 partner commits measurable audience/volume	Attribution and production readiness
Usage	Users complete entry and exit; repeat within 30/90 days	Funded retention and round-trip completion
Commercial	At least 1 accepted pricing or paid pilot	MRR/usage event and support economics

North-star metrics

- Product: funded wallets completing a reconciled capital round trip.
- SDK: production partner applications with completed workflows and attributable retained capital.
- Ecosystem: net-new sBTC deposits, lending supply/borrowing, swaps and active addresses attributable to tracked integrations.

Anti-metrics

Do not optimize for connected wallets without funding, gross quotes without signatures, submitted transactions without reconciliation, observed protocol TVL without attribution or headline APY without retained use and successful exits.

26. Delivery roadmap and team model

Milestone	Outcome	Primary proof
M0 Feasibility	Verified contracts, wallet paths, data access and protocol semantics	Capability matrix; real or test transaction spikes
M1 Foundation	Core types, registry, API, storage, ingestion and signing guards	Typed reads and deterministic unsigned plan guards
M2 Earn lifecycle	BTC -> sBTC -> supply -> withdraw -> sBTC workflow	Round trip survives reload/provider timeout
M3 Multi-protocol	Second lending, borrow/repay, swap, risk and optional staking	Normalized comparison without semantic loss
M4 Release	Embedded components, partner example, threat review, E2E, docs and pilot	Clean partner completes certified sandbox workflow

Two-engineer allocation

Engineer	Bias	Allocation
Kenzman	Higher technical depth	Core types, signing boundary, state machine, protocol adapters, risk arithmetic, threat review and release compatibility
IBK	Strong product/full-stack implementation	Platform scaffolding, database/API, ingestion worker, SDK client/hooks, product screens, embedded components, metrics, E2E and docs

Both engineers own SDK and product work. Cross-review is mandatory. The current implementation plan allocates 20 tasks and 40 estimated engineering days to each engineer, with a 10-week baseline and contingency for protocol/wallet integration uncertainty.

27. Major risks and architectural responses

Risk	Impact	Response
Protocol/API uncertainty	Planned route cannot execute	M0 feasibility; runtime capability gate; documented exclusion
Cross-chain delay or ambiguity	Duplicate deposit/withdraw action	Persistent evidence, nextAction and reclaim/support paths
Oracle/liquidity staleness	Unsafe borrow or misleading rate	Freshness limits and action-level disable
Normalization error	Incorrect value or risk comparison	Adapter-specific semantics, lineage and fixtures
Security incident	Funds or trust loss	Non-custodial scope, allowlists, audit and emergency flags
Yield compression	Weak user value	Sell orchestration, risk, reporting and distribution, not APY alone
Long partner sales cycle	Slow adoption	Sandbox, smallest embedded flow and clean reference app
Regulatory/commercial ambiguity	Launch or fee risk	Legal review before monetization; accurate disclosures

Open decisions before production

- First and second lending deployments, exact asset pairs and verified transaction methods.
- Approved swap router/contract access and enforceable exact-input constraints.
- Bitcoin Staking production capability and wallet support; default is disabled.
- RPC/indexer/provider quotas, historical coverage and fallback policy.
- Hosting, telemetry, final SLO targets and incident ownership.
- Fee/referral model, ranking disclosures and jurisdiction-specific legal approval.

28. Production acceptance checklist

Architecture

- One adapter passes read, quote, plan, sign, confirm and reconcile.
- Second adapter proves normalization without shared false assumptions.
- Production registry and transaction construction receive external review.

Product

- Deposit, Earn, Borrow, Repay, Swap and withdrawal show review, signature, submitted, confirmation and reconciliation states.
- Every multi-step flow resumes after reload without duplicate submission.
- Rates, balances, risk and health expose freshness, inputs and limitations.

Partner SDK

- Clean external app consumes published packages only.
- Webhook verification, deduplication, network switching and error recovery pass certification.

Operations

- Reorg, provider outage, signer delay, quote expiry and duplicate-write drills pass.
- Metrics, alerts, runbooks, backup restore and emergency action disable are proven.

Business

- At least one design partner completes a production-scoped pilot with attributable success metrics.
- Pricing, support ownership, terms, privacy and disclosures are approved.

Launch rule

The product launches the verified surface, not the promised surface. Any protocol, route or staking capability that cannot meet its acceptance evidence remains disabled without blocking reliable reads or safe exits.

29. Detailed engineering task allocation

The allocation is deliberately equal by estimated effort while asymmetric by technical risk. Kenzman carries the deepest protocol, transaction, financial-math and security responsibilities. IBK carries the platform, data, developer-experience and product implementation tracks. Dependencies create frequent integration points, and the non-owner reviews each pull request.

Engineer	Tasks	Estimated days	SDK / Platform / Product mix
Kenzman	20	40	Protocol adapters, core SDK, signing, state machine, risk, security and launch
IBK	20	40	Platform/API/data, public SDK/React, product UI, testing, operations and docs

Kenzman allocation - protocol and execution lead

ID	Gate	Work item	Dependencies	Days
K01	M0	Contract and market feasibility matrix	-	2
K02	M0	sBTC and staking feasibility spike	-	2
K03	M1	Core types and adapter contract	K01, K02	2
K04	M1	Signing boundary and transaction guards	K03	2
K05	M1	Reorg-aware ingestion design	K03	2
K06	M2	Persistent execution state machine	K04, I05	2
K07	M2	sBTC deposit adapter	K02, K06	2
K08	M2	sBTC withdrawal adapter	K07	2
K09	M2	First lending transaction adapter	K01, K04	2
K10	M2	Earn transaction integration review	K08, K09, I10	2
K11	M3	Second lending adapter	K09	2
K12	M3	Borrow and repay execution	K11	2
K13	M3	Swap routing adapter	K04, K01	2
K14	M3	Protocol risk arithmetic	K11, K12	2
K15	M3	Borrow UX safety integration	K12, I13	2
K16	M3	Staking adapter or documented exclusion	K02, K04	2
K17	M4	Transaction threat-model review	K10, K15, K16	2
K18	M4	Failure injection and recovery drill	K17, I17	2
K19	M4	SDK release compatibility gate	I16, K18	2
K20	M4	Pilot and launch decision	K19, I20	2

IBK allocation - platform and product lead

ID	Gate	Work item	Dependencies	Days
I01	M0	Data access and provider inventory	-	2
I02	M0	Wallet and product feasibility prototype	-	2
I03	M1	Monorepo and CI setup	K03	2
I04	M1	Database migrations and fixtures	K03, I01	2
I05	M1	API skeleton and tenant access	I04	2
I06	M1	Market ingestion and projection worker	I04, K05	2
I07	M2	SDK HTTP client and typed errors	I05	2
I08	M2	React hooks and cache isolation	I07	2
I09	M2	Wallet shell and portfolio screens	I02, I08, I06	2
I10	M2	Earn review and progress UI	I09, K06, K09	2
I11	M3	Position decoder and rewards projections	K11, I06	2
I12	M3	Earn comparison and rate breakdown	I11, I10	2
I13	M3	Borrow and repayment screens	K12, I08	2
I14	M3	Swap screen and quote expiry	K13, I08	2
I15	M3	Risk and activity screens	K14, I11	2
I16	M4	Embedded components and partner example	I12, I13, I14	2
I17	M4	Metrics, alerts and feature flags	I15, K14	2
I18	M4	Product end-to-end and accessibility tests	I15, I17	2
I19	M4	SDK docs and operator runbooks	I16, K18	2
I20	M4	Pilot support and release checklist	I18, I19	2

How the split works in practice

- Kenzman defines the invariant or protocol contract; IBK exposes it through API, SDK hooks and product states.
- IBK's data and UI work cannot declare success until Kenzman's adapter/state-machine evidence reconciles.
- Kenzman's execution work cannot release until IBK's partner example, E2E, accessibility, metrics and runbooks pass.
- M0 and M1 unblock parallel implementation; M2 proves one complete vertical slice; M3 expands capabilities; M4 hardens and certifies.
- Status moves Backlog -> Ready -> In progress -> In review -> Done, with Blocked and a recorded reason when a dependency or external capability stops progress.

ARCHITECTURE SUMMARY

One SDK. One execution model. Many distribution surfaces.

Everything Stacks proves the complete Bitcoin capital lifecycle. The Capital OS SDK turns those proven workflows into reusable infrastructure for wallets, fintechs, exchanges, custodians and treasuries. The system stays non-custodial, protocol-aware, evidence-driven and resumable across Bitcoin, Stacks and external provider failure.

Build	Prove	Distribute
Canonical data, adapters, quotes, plans, workflows and risk controls	Real deposit, earn, borrow, swap, repay and exit round trips	Partner SDKs, embedded components, webhooks and enterprise operations

End of technical architecture | Version 0.1